

Week 1

Week 1 Report – Hierarchical Clustering (Cosine Similarity)

Summary of Discussion

- Introduced **hierarchical clustering** for feature maps (FMs).
- Constructed a **proximity matrix** where each entry is the **cosine similarity** between two feature maps.
- At each step, select the **pair of FMs with the highest cosine similarity**.
- When merging two FMs, replace them with their **medoid** (the representative FM closest to all others in the cluster).
- This process is repeated iteratively, shrinking the number of feature maps.

Key Equations

- **Cosine Similarity:**
$$\cos(\text{FM}_i, \text{FM}_j) = \frac{\text{FM}_i \cdot \text{FM}_j}{\|\text{FM}_i\| \cdot \|\text{FM}_j\|}$$
$$\text{cos}(\text{FM}_i, \text{FM}_j) = \frac{\text{FM}_i \cdot \text{FM}_j}{\|\text{FM}_i\| \cdot \|\text{FM}_j\|}$$
- **Medoid Definition:**
The feature map FM_k is the medoid of a cluster if
$$\sum_{j \in \text{cluster}} d(\text{FM}_k, \text{FM}_j)$$
 is minimized (where d is a distance measure, e.g., cosine distance).

Decisions Made

- Use cosine similarity as the measure of closeness.
- Use the medoid instead of average when combining feature maps (to preserve interpretability).

Questions Left for Next Week

1. When to stop merging?

- If we keep merging until all FMs collapse into one cluster, we risk losing diversity and accuracy.
- Need a **stopping criterion** to balance efficiency and accuracy.

2. How to define similarity threshold?

- Should there be a cutoff similarity below which we stop merging?
 - How do we set this threshold meaningfully (not arbitrarily)?
-

My Answers / Reflections

1. When to stop merging?

- At this stage, no stopping rule was defined. But a natural idea is to stop when similarity between candidate pairs falls below a threshold.
- Another option: stop when merging reduces classification accuracy.

2. How to define threshold?

- Initially, one could try a **fixed cutoff** (e.g., similarity < 0.7).
- But this is arbitrary. Better methods include:
 - **Statistical approaches:** e.g., treat pairwise similarities as a distribution and stop merging when new merges fall below mean – 1σ .
 - **Validation-based approaches:** test clustering effect on held-out accuracy, stop when it no longer improves.

Week 2

Week 2 Report – Stopping Criteria in Hierarchical Clustering

Summary of Discussion

- In Week 1, we built the **cosine similarity proximity matrix** and performed hierarchical clustering with **medoid replacement**.
- The main limitation: **no stopping criterion** → clustering continues until all feature maps (FMs) collapse into a single cluster.
- Week 2 focused on finding a principled **stopping condition** to avoid two extremes:
 - **Too early stopping** → too many redundant FMs remain.
 - **Too late stopping** → important FMs may be over-merged, reducing accuracy.

Ideas Discussed

1. Threshold-based stopping:

- Stop merging when similarity between the most similar pair falls below a fixed threshold.
- But this is arbitrary and not principled.

2. Dissimilarity as criterion:

- Instead of only similarity, consider **dissimilarity** (cosine distance = $1 - \cos\theta$).
- Question: should merging stop when dissimilarity between best candidate pair exceeds some value?

3. Avoiding arbitrariness:

- The professor emphasized: thresholds are unavoidable in ML, but they should have **physical meaning** (e.g., probability-based, performance-based), not arbitrary.

- Otherwise, threshold = “toss a coin.”

Key Equations / Concepts

- **Cosine Distance (dissimilarity):**

$$d(\text{FM}_i, \text{FM}_j) = 1 - \frac{\text{FM}_i \cdot \text{FM}_j}{\|\text{FM}_i\| \cdot \|\text{FM}_j\|}$$

$$d(\text{FM}_i, \text{FM}_j) = 1 - \frac{\text{FM}_i \cdot \text{FM}_j}{\|\text{FM}_i\| \cdot \|\text{FM}_j\|}$$
 - **Stopping by threshold (tentative):**
 Stop when

$$d(\text{FM}_i, \text{FM}_j) > \tau$$
 for candidate pair, where τ is a threshold to be defined.
-

Decisions Made

- Clear need for a stopping rule before merging everything.
 - Decided that **threshold must be meaningful**, tied to probability of correction, performance, or statistical testing — not an arbitrary constant.
 - This left the team at a “pause point”: stopping criterion identified as the **key next step**.
-

Questions Left for Next Week

1. **How do we define a “meaningful” threshold?**
 - Should it be tied to accuracy improvement?
 - Should it be tied to error correction between FMs?
2. **What is the role of dissimilarity vs similarity?**
 - Should stopping be based on lack of similarity or on lack of correction benefit?
3. **Can we borrow from statistics?**

- For example, use **T-tests** or distributional cutoffs to decide whether two FMs are “significantly similar/dissimilar.”
-

My Answers / Reflections

1. Threshold definition:

- Instead of a raw similarity threshold (e.g., cosine > 0.7), a **probabilistic correction-based threshold** is better.
- Example: Only keep FM2 if it **corrects a significant proportion of FM1's errors**.

2. Dissimilarity vs correction:

- Dissimilarity alone is not enough; two FMs can be dissimilar but redundant (e.g., both wrong in different ways).
- Correction ability captures whether FM2 actually adds value on top of FM1.

3. Statistics-based approach:

- Using **T-test or confidence intervals** would make stopping criterion less arbitrary.
 - Could frame it as: “Are FM1 and FM2's correction abilities significantly different?”
 - If not, merge them. If yes, keep them separate.
-

Takeaway from Week 2

- **Problem identified:** When to stop merging?
- **Progress:** Realized the need for a threshold with **physical justification**.
- **Next step (Week 3):** Explore correction-based stopping → compute probability of one FM correcting another's mistakes and use that as the merging/stopping criterion.

Week 3

Week 3 Report – Correction-Based Stopping in Hierarchical Clustering

Summary of Discussion

- Week 2 ended with the open problem: *how to set a meaningful stopping criterion for clustering*.
- Week 3 introduced a **new idea**: use **correction ability between feature maps (FMs)** as the basis for deciding whether to merge, drop, or keep FMs.

Key Ideas

1. Correction Concept

1. FM2 “corrects” FM1 if, when FM1 makes an error, FM2 gives the true class.
2. This is asymmetric: FM2 correcting FM1 is not the same as FM1 correcting FM2.

2. Correction Factor

1. Probability that one FM corrects the other’s mistakes.
2. Captures whether an FM is **redundant, random, or useful**.

3. Decision Options

At each merge step, for candidate pair (FM1, FM2), we can:

1. **Keep both** – if they provide distinct correction value.
2. **Drop one** – if one is redundant (correction ≈ 0).
3. **Combine into a medoid** – if both provide nearly the same value (correction factors similar).

Key Equations / Metrics

- **Correction Probability:**

$$P(\text{FM2} = \text{C2} | \text{FM1} = \text{C1}, \text{TC} = \text{C2}) / P(\text{FM2} = \text{C2} | \text{FM1} = \text{C1}, \text{TC} = \text{C2})$$
 where TC = true class.
- **Benefit of FM2 over FM1:**

$$\text{Benefit}(\text{FM2}) = \text{Err}(\text{FM1}) \times \text{CorrectionRate}(\text{FM2} | \text{FM1})$$

$$\text{Benefit}(\text{FM2}) = \text{Err}(\text{FM1}) \times \text{CorrectionRate}(\text{FM2} | \text{FM1})$$
- **Example Calculation:**
 - FM1 accuracy = 90% → error rate = 0.1
 - FM2 corrects 70% of FM1's errors → correction = 0.7
 - Benefit of FM2 = $0.1 \times 0.7 = \mathbf{0.07}$ (7% gain).
- Similarly:
 - FM2 accuracy = 80% → error rate = 0.2
 - FM1 corrects 40% of FM2's errors → correction = 0.4
 - Benefit of FM1 = $0.2 \times 0.4 = \mathbf{0.08}$ (8% gain).
- → We now have **two asymmetric correction values**: 0.07 and 0.08.

Decisions Made

- Correction factor is a **meaningful metric** for deciding whether two FMs should be merged, dropped, or kept.
- Merging decisions should not be based solely on similarity (cosine), but on whether one FM actually adds predictive power to another.

Questions Left for Next Week

1. What numeric threshold should be used for correction factor?

- Is 0.5 (random chance baseline) a reasonable cutoff?
- Or should smaller/larger thresholds be tested?

2. **Should statistical tests be used?**

- For example: T-test to see if correction factor is significantly better than chance.
- Or treat correction factors across pairs as a distribution and merge if values are within 1σ of each other.

3. **Scalability:**

- How does this apply when comparing hundreds of FMs (e.g., 512 from Conv2)?
 - Do we evaluate correction pairwise for all, or approximate somehow?
-

My Answers / Reflections

1. **Threshold setting:**

- A raw 0.5 threshold has some justification (random chance baseline), but may be too crude.
- A **data-driven threshold** (e.g., mean or median correction rate across all pairs) could adapt better.

2. **Statistical testing:**

- Yes, useful: a T-test or confidence interval can avoid arbitrary cutoffs.
- If correction rates between FM1 and FM2 are not significantly different, we can merge.

3. **Scalability:**

- Full pairwise correction matrix (512×512) may be computationally heavy.
- Possible shortcut: only compute correction for **nearest neighbors in cosine similarity**, not all pairs.

Takeaway from Week 3

- **Breakthrough:** Correction factor gives a principled way to decide merging/stopping.
- **Remaining Gap:** Still no finalized numeric threshold → that's the next empirical step.
- **Next Step (Week 4):**
 - Experiment with different thresholds (0.3, 0.5, 0.7).
 - Compare fixed vs statistical thresholds.
 - Evaluate impact on accuracy and number of surviving FMs.

Week 2 → Week 3: How Concerns Were Addressed

Concern 1 (Week 2): Stopping Criterion

- **Week 2 issue:** Hierarchical clustering doesn't tell us *when to stop combining*. Using cosine similarity alone was risky because highly dissimilar but discriminative FMs might get combined incorrectly.
 - **Week 3 response:**
 - Instead of raw similarity, introduce a **functional criterion** — the *correction probability* → refined into the *Benefit metric*.
 - This grounds merging in *classification contribution*, not just geometry.
 - So, “when to stop” = when no FM provides a meaningful benefit over another.
 - **My suggestion:** This is a **big improvement**. It ties stopping directly to predictive power rather than arbitrary thresholds. But:
 - It's still vulnerable to **data imbalance** (e.g., if some classes have far fewer samples, correction probability may be unstable).
 - A safeguard might be to normalize correction counts per class or use weighted correction.
-

Concern 2 (Week 2): Threshold for Combining

- **Week 2 issue:** How to decide whether two FMs should be merged (e.g., arbitrary 0.5 cutoff was mentioned but seen as weak).
 - **Week 3 response:**
 - Correction probability alone replaced by **Benefit = Correction % × Error rate**.
 - This prevents “illusory benefit” when the first FM makes very few errors.
 - Still leaves the *how close is close* problem → suggested statistical testing (T-test) to decide if benefit differences are significant.
 - **My suggestion:**
 - T-tests are principled, but repeated testing risks false positives (error accumulation).
 - A lighter alternative: use **distribution-based thresholds** (mean ± std. dev of benefit differences across pairs) — avoids arbitrary cutoffs without heavy testing.
 - This could be presented as: *“Professor, perhaps instead of repeated T-tests, we could consider a distributional cutoff to avoid error accumulation?”*
-

Concern 3 (Week 2): Accuracy as Feedback is Expensive

- **Week 2 issue:** Using classification accuracy for feedback is too expensive at clustering time. Entropy was considered as a proxy.
- **Week 3 response:**
 - Accuracy wasn’t used directly. Instead, **correction-based benefit** implicitly measures discriminative contribution *without retraining/testing the model*.
 - So the move from entropy → correction probability → benefit is essentially a more tailored proxy for accuracy.
- **My suggestion:**

- This was well addressed — the benefit metric is much more interpretable and cheaper to compute.
 - But maybe mention explicitly that “benefit” is **like entropy** in spirit — a proxy for accuracy — but tuned for feature map pruning. That might score points for theoretical connection.
-

Concern 4 (Week 2): Risk of Combining Dissimilar but Important FMs

- **Week 2 issue:** Two highly dissimilar FMs may both be critical for classification (small cosine similarity doesn’t mean redundancy).
 - **Week 3 response:**
 - Symmetric benefit measure ensures both FMs are checked against each other. If both contribute significantly to correcting errors, they won’t be merged or dropped.
 - **My suggestion:**
 - This is solid. The symmetry explicitly fixes Week 2’s concern.
 - I’d highlight this in discussion: *“The symmetry of the benefit metric is the key reason it resolves Week 2’s dissimilarity concern.”*
-



Overall Assessment

- The **Week 3 refinements directly answer Week 2’s open problems.**
- The **benefit metric** is a much stronger theoretical foundation than raw similarity or entropy alone.
- **Remaining weak point:** The *threshold problem* (how close is close) still lacks a universally solid solution.
 - Current proposal: T-tests → good in theory, but error accumulation risk.
 - Suggested addition: distribution-based cutoff (lighter, avoids multiple testing problem).

Week 4

Week 4 Report

1. Core Method (Common Ground)

The foundation of the project is now stable and agreed upon:

- **Similarity metric:** Cosine similarity (preferred due to stability and interpretability).
- **Clustering method:** Hierarchical clustering (HC), bottom-up.
- **Stopping rule:** Threshold criterion (cosine ≥ 0.9 used as elimination cutoff).

These components remain **shared between both students**. The differentiation in contributions starts **after clustering**, during merging, handling redundancy, and backpropagation strategy.

2. Points of Differentiation

(a) Merging Strategy

When feature maps (FMs) are highly similar (cosine $\geq$ threshold), pruning requires a merging step.

- **Student A:** Median (midoid).
- **Student B:** Average (mean).

Weighted averages were considered but rejected: since feature maps are usually close in contribution, unequal weights are difficult to justify.

(b) Handling Redundancy (Nearest Neighbor Assumptions)

To address redundancy and co-linearity:

- **Student A:** Pure 1-nearest neighbor (1-NN).

- **Student B: 1 + 2 nearest neighbors** (extend beyond top-1).

This ensures balanced contributions and satisfies the requirement for individual work (~20% difference).

3. Co-linearity Challenge (Deferred)

- **Problem:** Highly correlated feature maps across layers.
 - **Options discussed:**
 - Pairwise correlations (infeasible due to combinatorial explosion).
 - Nearest-neighbor-based simplifications.
 - Possible visualization of correlation impact.
 - **Decision:** This is a **very hard problem**. Set aside for now to avoid derailment. May return to it later if time permits.
-

4. Backpropagation Challenge (Next Major Focus)

Problem: After pruning FMs at one layer, how does this affect the next layer's feature maps?

Strategies from Literature:

1. **Fine-tuning after pruning** (small sample) → recalibrates weights, mitigates error accumulation.
2. **Batch Normalization (BN) recalibration pass** → forward pass on small subset, reduces noise without full training.
3. **Independent layer assumption** → prune each layer independently, assume self-calibration.
4. **Global pruning** (all layers jointly, e.g., Centroid Kernel Alignment) → complex, less favored.

Professor's preference: Local, **layer-by-layer pruning with possible fine-tuning**, rather than global methods.

Backpropagation strategy may also serve as a **future differentiation point** between the two students.

5. Current Implementation Status

- Code already implemented for small-scale runs (~1k samples on Colab).
 - Early results:
 - 10–25% FM reduction observed.
 - Limited due to capped number of tested pairs (2.5k vs full 130k).
 - Full-scale experiments expected to yield **significantly higher pruning rates**.
 - Time bottleneck mostly from statistical testing — this has been dropped in favor of the threshold rule, which is more efficient.
-

6. Next Steps (for Week 5)

1. **Both students:**
 - Share base code.
 - Branch into two paths:
 - Student A → Median merging + 1-NN assumption.
 - Student B → Average merging + 1+2-NN assumption.
2. **Research backpropagation strategies:**
 - Fine-tuning vs BN recalibration.

- Independent vs layer-coupled pruning.
 - Document strengths, trade-offs, and possible points of difference.
3. **Prepare results** comparing pruning rates and accuracy impacts under both merging strategies.
-

7. Key Takeaways for Week 4

- **Balance achieved:** Two points of difference established (merging strategy, nearest-neighbor assumption).
 - **Co-linearity:** Too complex → deferred.
 - **Backpropagation:** Identified as **next central research focus**.
 - **Implementation:** Early tests promising; larger runs needed.
-

✓ This Week 4 report now flows as one clean document and captures **all four meeting parts (1–4)** without repetition.

Stage	Common Ground	Student A	Student B (You)	Pros & Cons (Differentiation Justification)
1. Similarity Calculation	Cosine similarity for FM pairs	Same as B	Same as A	✓ Agreed as standard in research, no point of difference.
2. Clustering Framework	Hierarchical clustering (bottom-up, merge most similar FMs first)	Same as B	Same as A	✓ Both aligned. Efficiency + accepted method.
3. Stopping Criterion	Benefit-based ratio (min/max $\geq 0.9 \rightarrow$ "close")	Uses 0.9 threshold as is	Same	✓ Arbitrary threshold acknowledged, but both must use it for comparability.
4. Handling "Close" FM Pairs	If benefits are very close (≥ 0.9 ratio)	Medoid combination (median across activations)	Average combination (mean across activations)	♦ A's <i>Medoid</i>: more robust to outliers. ♦ B's <i>Average</i>: simpler, smoother, less noisy.
5. Handling Redundancy (Non-Close)	If one FM clearly dominates (ratio < 0.9) $\rightarrow$ drop weaker	Drop weaker FM	Drop weaker FM (same)	✓ Both aligned; no difference here.
6. Co-linearity (Correlated FMs)	Must address correlation problem somehow	1-Nearest Neighbor assumption (FM is only correlated with strongest partner)	2-Nearest Neighbor or Extended k-NN approach (FM may be correlated with top 2 or few partners)	♦ A's 1-NN: fast, simple, but may miss wider correlations. ♦ B's k-NN: captures richer structure, but higher compute cost.

7. Alternative Merge Operations	When combining FMs	Uses Median (Medoid)	Uses Mean (Average)	◆ Differentiates statistically: median vs mean trade-off.
8. Backpropagation Strategy (Future Work)	Must handle how pruning one layer affects next	Will explore fine-tuning with small sample after pruning	Will explore independent layer assumption (no fine-tuning, rely on self-calibration)	◆ A's fine-tuning: safer, preserves accuracy, but more compute. ◆ B's independence: faster, cheaper, riskier.
9. Visualization of Co-linearity	Both need to measure how big problem is	Bar chart of FM benefits only	Will explore distribution of correlations (approx. k-nearest, instead of all pairs)	◆ A = simpler, but ignores correlation scale. ◆ B = better diagnosis of redundancy overlap.

Week 5

Here's a structured **summary of Week 5, Part 1** discussion based on what you shared:

Week 5 Part 1 Summary

Main Points Covered

1. Cosine + HC as the baseline approach

- Reiterated that cosine similarity + hierarchical clustering (HC) remains the core method.
- The **colinearity problem** (highly correlated FMs) was acknowledged but set aside for now, since it's difficult to tackle directly.

2. T-test vs. thresholding

- Earlier code implementation used T-tests on a small sample (1000 images).
- Switching to a **fixed threshold (0.9 cosine similarity)** brought major improvements:
 - **30–40% faster runtime** (since T-tests are skipped).
 - FM pruning increased from ~25–30% to an expected **70%+** with larger coverage.

3. Backpropagation strategies (research findings)

- **Sequential pruning**: prune layer by layer, starting from the last layer and moving backwards.
- **Interleaved pruning**: prune partially across multiple layers in small steps, alternating between them to avoid destabilizing one layer too much.
- Both strategies are widely discussed in the literature.

4. Alternative pruning refinements

- **Slice/channel-wise pruning:** Instead of pruning entire feature maps, prune by slices or channels to retain finer-grained structure.
- **Conditioning on set S (kept FMs):** When calculating benefits, condition on the set of already-kept FMs, making decisions relative to what's retained. This reduces redundancy more effectively.

5. Branching out between Student A and B

- Student A: median merging + 1-NN.
- Student B: mean merging + k-NN extension.
- Professor encouraged both to keep reading literature to avoid “reinventing the wheel.”

6. Hybrid approaches (Student B idea)

- Instead of a single fixed threshold, use **multiple thresholds**:
 - At very high similarity ($\geq 95\%$): drop the weaker FM.
 - At high similarity ($\sim 90\%$): merge via averaging.
 - At moderate similarity: consider partial merging or weighted averaging.
- This was not fully clear yet in terms of implementation but is promising as a way to push the limits and compare against Student A's safer method.

7. Sparsity issue in FMs

- Sparse feature maps look artificially “similar” under cosine/Pearson measures.
- Recognized as unavoidable—similarity metrics alone cannot resolve it.
- Some structural similarity methods exist, but they are too computationally expensive.

8. Benefit recomputation & storage S

- Each branch of the HC maintains its own **local set S** of retained FMs.
- After merges, benefits may need recalculation:

- Doing it after every merge is expensive.
 - **Batch recomputation by round** is a practical compromise.
-

✓ Agreed Directions for Next Steps

- Stick with **cosine + HC + thresholding** as the main pipeline.
- Investigate **backpropagation strategies** (sequential vs interleaved).
- Implement **branching differences** (median vs mean, 1-NN vs k-NN).
- Explore **hybrid thresholds** and partial merging (Student B's angle).
- Batch recomputation of benefits for efficiency.

Here's a structured summary of **Week 5 – Part 2** discussion you shared:

Key Focus

- Discussion on **conditioning on set S** during pruning iterations.
 - Exploration of whether **recomputations after every merge** are necessary vs. batching updates.
-

Main Points

1. Iteration Setup

- Start with feature map comparisons (FM1 vs FM2).
- Initially, **S** = $\emptyset$ (null set).
- Decisions: **merge or drop** → update S accordingly.

2. Effect of Conditioning on S

- After a merge (e.g., FM1 + FM2 → merged), **S is updated**.
- In the next iteration (FM3), benefits are computed as **FM3 | S**.
- Concern: this requires recomputation whenever S changes → **computationally expensive**.

3. Is Work Duplicated?

- No: benefits for FM3 weren't computed earlier, so computing them when needed isn't duplication.
- But **frequency of recomputation** remains an issue.

4. Proposal: Batch Updates ("Per Round")

- Instead of recomputing after every merge, **stack merges/drops** for a round.
- After applying a set of merges, **recompute statistics once per round**.
- Idea: fewer recomputations, reduced cost.

5. Professor's Counterpoint

- Without updated statistics, you **cannot make reliable decisions** (merging needs updated benefits).
- Batching may skip critical comparisons, leading to **wrong pruning decisions**.
- Example: if FM3 was dropped in round 1, comparisons FM3 vs FM4 never occur—so batching doesn't save effort, since drops already reduce comparisons.

6. Alternative Case Concern

- When comparing FM1 vs FM3:
 - Need both **Benefit(FM1 | FM3)** and **Benefit(FM3 | FM1)** before deciding.
 - Cannot shortcut this by only considering one direction.
- Once FM3 is dropped, no further conditionals with FM3 are needed.

- Thus batching doesn't actually eliminate necessary computations.

Consensus / Outcome

- **Conditioning on S is unavoidable** and recomputation must occur after merges to ensure correctness.
- Batching (per round) looks attractive, but in practice it:
 - Doesn't reduce valid comparisons.
 - Risks **losing correctness** by delaying updates.
- Final takeaway: **stick with per-iteration updates**, as batching offers little real benefit.

✅ So essentially, Part 2 was about debating if batching updates to S can save computation. The conclusion was that while it sounds efficient, in reality, it risks correctness and doesn't reduce necessary work.



Student A vs Student B: Comparison

Aspect	Student A (Single-threshold, batching idea, cosine)	Student B (Hybrid, multiple thresholds, Pearson)
Similarity metric	Cosine similarity (no centering). Captures angular closeness but can overestimate correlation if magnitudes differ.	Pearson correlation (centered). Removes mean bias, captures linear relationships more faithfully → stronger for redundancy detection.
Redundancy test	Single threshold (ratio ≥ 0.9) plus magnitude gate ($m \geq 0.02$). If FMs are nearly equivalent, prune/merge.	Multiple thresholds (hybrid): uses tiers: • High ratio + high magnitude → drop • Medium ratio + moderate magnitude → merge • Low ratio but non-trivial magnitude → keep both (avoid over-pruning).
Decision rule	Deterministic: if ratio passes, prune the weaker FM. Otherwise keep both.	Flexible: different thresholds for drop, merge, keep. Allows fine-grained tradeoff between accuracy and compression.
Statistical safeguard	McNemar's test used optionally; professor preferred ratio because simpler.	Hybrid: thresholds act as soft guardrails, and statistical tests can be fallback checks if ambiguity remains.
Batching	Proposed batching merges per round (compute similarities once, apply multiple merges, then recompute). Professor was skeptical about correctness.	Does not rely on batching. Still incremental like original method, but applies multi-threshold logic at each decision point.
Merge operator	Drop, average, medoid (depending on config).	Same merge ops, but thresholding decides when merge is preferable over drop.
Stopping rules	Accuracy guard + IC stopping.	Same, but hybrid thresholds may reduce reliance on stopping rules (since decisions are more conservative).
Complexity / FLOPs	Lower complexity (only one threshold to check). Risk: more aggressive pruning → potential accuracy drop.	Slightly higher computation (checking multiple thresholds), but avoids over-pruning. Better tradeoff between FLOPs reduction and accuracy.

Risk profile	Simpler, faster to implement, but brittle — one global threshold may fail across all layers and datasets.	More robust: adapts decisions depending on context. Risk: more parameters to tune (threshold values).
Example (FM1 vs FM2)	Ratio = 0.92, m = 0.03 → drop weaker FM. If ratio = 0.88, keep both.	Ratio = 0.92, m = 0.03 → drop. Ratio = 0.88, m = 0.05 → merge instead of discarding. Ratio = 0.60, m = 0.10 → keep both (since both contribute uniquely).

🔑 **Positioning for Professor**

- Student A: **“clean, simple, single-threshold pipeline”**. Strong baseline.
- Student B (you): **“robust hybrid refinement”** that:
 - Uses Pearson instead of cosine (better for redundancy).
 - Adds **multiple thresholds** to balance drop vs merge vs keep.
 - Sacrifices a little simplicity for **better accuracy vs FLOPs tradeoff**.

Ah! Thanks for clarifying — you’re right, we need to be precise here:

- In **your approach**, the similarity metric **is set to Pearson** (SIM_METRIC="pearson"). Student A uses Cosine by default.
- You **do explicitly apply multiple thresholds** (e.g., >0.95, 0.85–0.95, <0.85), so the “single threshold” description does **not** apply to your run.

Updating the table with that correction:

Feature / Policy	Student A	Your Approach (current run)	Notes / Difference
------------------	-----------	-----------------------------	--------------------

Similarity metric	Cosine	Pearson	Explicit centering; differs from Student A
Thresholds for pruning	Single (0.9)	Multiple ranges (>0.95, 0.85–0.95, <0.85)	Explicit multi-threshold approach
Statistical tests (McNemar)	Not used	Implemented but inactive	Only computed if <code>DECISION_POLICY="stats"</code> is set
Magnitude gate (m)	Not used	Used (<code>m >= BENEFIT_MAG_MIN</code>)	Ensures non-trivial effect
Merge ops	drop / avg / medoid	drop / avg / medoid	Same
IC stop / Accuracy guard	Optional	Optional	Same
Greedy disjoint scheduling	Yes	Yes	Same
Visualization titles include r & m	No	Yes	Added for professor clarity

Key clarifications to state:

1. Your pruning run uses **Pearson similarity** instead of Cosine.
2. You **explicitly implement multiple thresholds**, unlike Student A.

3. **Statistical tests are implemented but not used**; pruning depends on ratio + magnitude gate.

If you want, I can also **highlight in the code exactly where each of these differences occurs**, so it's very easy for the professor to see. Do you want me to do that?

Code Summary

Cell 1:

Got it 👍 thanks for sharing the **first cell**. Let's go step by step.

✅ What this cell does

- Imports required libraries (`torch`, `numpy`, `PIL`, etc.).
 - Defines `CONFIG` dictionary with **hyperparameters** (epochs, batch size, dataset size).
 - Sets up **professor's specific method parameters**:
 - `IOU_THRESHOLDS`: `[6, 7, 8, 9]`
 - `STATISTICAL_ALPHA`: `0.05` (t-test sig. level)
 - `DELTA_TOLERANCE`: `1e-3` (stabilization threshold)
 - Seeds for reproducibility.
 - Device setup (CUDA/CPU).
 - Helper functions (`seed_worker`, `find_image_dir`).
 - Sanity-check for transforms.
-

🔄 Should you change anything for your method?

Yes — only in the **configuration block**.

This cell currently reflects **professor's method** (IOU + statistical testing).

Since your contribution is **Pearson correlation + hybrid multiple thresholds**, you need to:

- Remove IOU and statistical test configs.
- Add your similarity + thresholds configuration.

Everything else (imports, seeding, device setup, helper functions) can stay the same.

Key Differences vs Student A:

- Replaced "IOU_THRESHOLDS" and "STATISTICAL_ALPHA" with "CORR_THRESHOLDS" and "MAG_THRESHOLDS".
- Config now reflects **your method** (Pearson + hybrid thresholds).
- Next steps updated to match your approach.

👉 So for this **first cell**, you **must change the CONFIG block** (and related prints). Everything else can remain identical.

Cell 2:

Perfect, thanks for sharing **cell 2**. Let's break it down:

What this cell does

- Defines the **baseline CNN architecture (CNN_V1)**.
 - Structure:
 - 5 convolutional blocks (Conv → BatchNorm → ReLU → MaxPool).
 - After 5 pools, feature maps are downscaled → flattened → fully connected classifier.
 - Dropout included for regularization.
 - Final classifier outputs `num_classes = 2` (male vs female).
 - Instantiates model and prints parameter count.
-



Do you need changes for your method (Student B)?

No.

Here's why:

- The **CNN backbone** is **independent** of the pruning/similarity method.
- Both you (Student B) and Student A start from the **same baseline CNN** before applying pruning.
- Your method (Pearson + multiple thresholds) only comes into play **after feature extraction**, when analyzing **layer 5 feature maps**.

So, **you can use this cell exactly as is**.



My recommendation

- Keep this cell **unchanged** for your implementation.
- But you may want to add a comment clarifying that this is the **shared baseline** for both Student A and Student B.

👉 So for **cell 2**, no modifications needed for Student B — only the comments/annotations if you want clarity in your submission.

Cell 3:

Great, thanks for sharing **cell 3**. Let's analyze it carefully:



What this cell does

- Defines a function `init_weights(m)`:
 - For each **Conv2d** or **Linear** layer:

- Initializes weights with **He/Kaiming initialization** (good for ReLU activations).
- Initializes biases to zero.
- Applies this initialization to your model with `model.apply(init_weights)`.

This ensures **stable training** and avoids exploding/vanishing gradients.

Do you need changes for your method (Student B)?

No.

Here's why:

- This cell is **completely independent** of pruning or feature map similarity (IOU vs Pearson).
- Both Student A and Student B's methods still train the **same baseline CNN** before pruning.
- The initialization strategy is not tied to the pruning method.

So, you can **reuse this cell exactly as it is**.

My recommendation

- Keep this cell unchanged.
- (Optional) Add a short clarifying comment that **initialization is shared across both approaches**.

👉 So, for **cell 3**, same as cell 2: **no modifications needed for Student B**.
We only diverge from Student A later (feature extraction + pruning phase).

Cell 4:

Perfect — let's walk through **cell 4** in detail.

What this cell does

1. **Installs Kaggle CLI** in Colab and authenticates using your `kaggle.json` API key.
 - Needed for downloading datasets directly from Kaggle.
2. **Downloads the CelebA dataset** (`jessicali9530/celeba-dataset`).
3. **Unzips** the dataset into `/content/celeba`.
4. **Updates CONFIG** to point to this dataset.
5. Leaves a note about reusing the custom loader.

This ensures you have **CelebA images locally available** for your CNN training and later feature extraction.

Do you need changes for your method (Student B)?

No changes are needed.

Here's why:

- Student A and Student B both train/test on the **same dataset**.
- Your contribution (Pearson + multiple thresholds) only affects the **feature map similarity and pruning phase**.
- The **dataset preparation step** is agnostic to the pruning method.

So, you can keep this cell exactly the same.



My recommendation

- Keep the cell unchanged.
- Add a clarifying comment that this dataset setup is **shared across Student A & B**, to highlight fairness of comparison.



So far (cells 1–4), **everything is the same** between Student A and B.

The **first real differences** will appear when we get to **feature map similarity + pruning logic** (Professor's vs your approach).

Cell 5:

Great — thanks for sharing **Cell 5**. Let's carefully unpack this one.



What this cell does

This is the **data pipeline setup**. It:

1. **Finds the CelebA image directory** and verifies metadata CSVs.
2. **Reads attributes + partitions**, merges into a single dataframe.
3. **Handles sampling**:
 - `train_df_random` → random subset (for CNN training).
 - `train_df_sequential` → ordered subset (for IOU analysis in Student A/Professor's method).
 - `valid_df` → validation subset.
4. **Defines transforms** for preprocessing images.
5. **Creates a custom PyTorch Dataset** (`CelebACustom`) for gender classification.
6. **Builds DataLoaders**:
 - `train_loader` → shuffled, for training CNN.

- `valid_loader` → validation.
- `sequential_loader` → no shuffle, for IOU analysis (Professor's).

7. Runs a **smoke test** to check batch shapes, label distribution, etc.

Student B — Do you need changes?

- **Dataset setup (CelebA gender classification)** →  Same.
- **Random sampling for training** →  Same.
- **Sequential dataset** →  Not directly needed for your Pearson + multiple threshold method.
 - You won't need a dedicated sequential loader, because Pearson correlation can be computed from activations regardless of dataset ordering.
 - That said, you *can* keep it for fairness in comparison, since the professor may want consistency.

👉 **So:** Keep most of this cell as-is, but you can clarify that the **sequential loader is unused in Student B's method**, only in Student A/Professor's.

Key takeaway

- For **Student B's Pearson + multiple thresholds**, you **don't need sequential order** → but you can leave it in place for fairness and direct comparison with Student A.
- No major rewrites are needed, just clarifications in comments.

Cell 6:

Perfect — thanks for sharing **Cell 6 (Smoke Test)**. Let's break it down carefully.

What this cell does

This is a **sanity check** to make sure:

1. **Model + DataLoader compatibility** → shapes match, model runs forward pass without errors.
2. **Loss function works** → checks gradient flow with `CrossEntropyLoss`.
3. **Performance check** → forward and backward pass timings.
4. **Debug feature timings** → measures per-layer compute time in the CNN's `features` block.

This is a **shared utility** — neither Student A's IoU method nor Student B's Pearson method requires changes here.

Student B — Do you need changes?

- **Loss function:** `CrossEntropyLoss` → correct, since this is still binary (0/1) classification but we treat it as a 2-class softmax problem. ✓
- **Output shapes:** Model produces logits `[batch, 2]`. ✓
- **Timing:** Works for both methods (since both build on the same CNN backbone). ✓
- **Sequential loader:** Not used here (that was for Student A). This test typically uses `train_loader` or `valid_loader`.

👉 So **no changes are required** for Student B. The comments and function names remain valid.

✓ This cell works **unchanged** for you (Student B).

I only added clarifying comments that it is **shared** and not tied to either Student A or Student B's method.

Cell 7:

Nice — **Cell 7** is just about setting up directories before training begins. Let's carefully place it in your Student B context.

✓ What this cell does

- Defines **output directories** for:
 - `LOG_DIR` → training/validation logs (loss, accuracy, etc.).
 - `CKPT_DIR` → model checkpoints (weights).
 - Organizes outputs **by phase** (so Phase 1 logs/checkpoints don't overwrite Phase 2, etc.).
 - Updates `CONFIG` dictionary so downstream training code automatically saves to the right place.
-

↺ Student B — Any changes needed?

- **No**. This cell is independent of the evaluation method (IoU vs. Pearson correlation with multiple thresholds).
- Both students rely on the same training loop and checkpointing.
- The **divergence happens only after model training** when you analyze redundancy (Student A with IoU, Student B with Pearson multiple thresholds).

⚡ Key takeaway:

This is **pure setup**. It doesn't touch model logic, data pipeline, or redundancy handling — so it works exactly the same for you (Student B) and Student A.

Cell 8:

Perfect — Cell 8 is the **core training loop**. Let's walk through it carefully in the context of **Student B (Pearson + multiple thresholds)**.

✓ What this cell does

- Defines `train_and_evaluate()`:

- Trains the CNN using **Adam** optimizer + **CrossEntropyLoss**.
 - Tracks loss & accuracy for training and validation.
 - Logs results (CSV).
 - Saves the **best checkpoint** (highest validation accuracy).
- Uses **run_smoke_test** first to confirm model + dataloader compatibility before training.
-

Student B — Any changes needed?

- **No changes to training.**
Your Pearson multiple-threshold method applies **after training**, when you evaluate redundancy among feature maps.
- Student A also trains the same way, because redundancy analysis (IoU vs. Pearson) doesn't affect gradient updates, optimizer, or loss.

So this code remains **100% reusable**.

⚡ Key takeaway:

- **Training loop stays identical** for both Student A and Student B.
- The **divergence begins after training**, when feature maps are analyzed:
 - Student A → IoU thresholding.
 - Student B → Pearson correlation + multiple thresholds (hybrid).

Cell 9 & 10:

Perfect, now I see the situation clearly. 

You pasted **two versions**:

- **Earlier Phase-3 (v3.0)** — where redundancy decisions were based on *similarity + either stats or ratio*.

- **New Phase-3 (v4.0)** — professor-aligned, adding the **benefit magnitude gate (m)** and making **Cosine vs Pearson** explicit.

Are they for the same purpose?

Yes — **both versions aim to prune redundant channels in the final conv block** while preserving accuracy.

The **core purpose is identical**:

- Identify similar channels (via activations).
- Decide whether to **merge/drop** based on redundancy tests.
- Perform channel pruning, update Linear layer, and check FLOPs/params/accuracy.

Why changes were made in v4.0

Area	v3.0	v4.0	Why the change
Similarity metric	Had a knob (<code>COSINE_CENTER</code>) that blurred cosine vs Pearson.	Explicit switch: <code>SIM_METRIC="cosine"</code> or <code>"pearson"</code> .	Clarity + avoids ambiguity.
Redundancy test	Used ratio (<code>min/max</code>) or stats (McNemar).	Adds m = (n01+n10)/N as magnitude gate (default ≥ 0.02).	Prevents “equivalence” from trivial cases with tiny disagreements.
Decision policy	Ratio @ 0.9 or stats.	Still ratio/stats, but always gated by m .	Stronger safeguard.

Dominance rule	Based only on ratio & strengths.	Explicit DOMINANCE_MIN + ratio.	Ensures dominance is not declared on vanishingly small effects.
Visuals	Showed similarity + ratio/p-values.	Adds m in titles.	Transparency for professor.
Config clarity	Threshold tuning less structured.	Clear config: SIM_METRIC , RATIO_NEAR_EQ , BENEFIT_MAG_MIN .	Matches professor's Week-6 plan.

So → **same pruning purpose**, but v4.0 = more **statistically disciplined and professor-aligned**.



Your task

You want:

- **Keep v4.0 codebase intact** (since it's professor-aligned).
- Modify only the **redundancy decision logic** to reflect **your approach (Student B: Pearson + multiple thresholds / hybrid)**.

That means:

1. **SIM_METRIC="pearson"** (instead of cosine).
2. In the decision step: allow **multi-threshold hybrid**:
 - **High threshold (strict ratio ≥ 0.95)** → safe prune.
 - **Medium threshold (0.85–0.95)** → keep only if magnitude **m ≥ 0.05** .

- **Low threshold (0.75–0.85)** → accept only if BOTH $m \geq 0.1$ and $p\text{-value} \geq \alpha$.
 - Otherwise → reject.
-

👉 This way, you and the other student (A) can run **the same v4.0 framework**.

- You: set `SIM_METRIC="pearson"`, `DECISION_POLICY="hybrid"`.
- Student A: set `SIM_METRIC="cosine"`, `DECISION_POLICY="ratio"`.